

Attention, Seq2Seq, Generative Text

Category : ATTENTION BASICS

Points : 100

Q1: What does TF-IDF stand for, and why is it considered a form of "static" attention?

A: Term Frequency - Inverse Document Frequency. It is "static" because the weights are fixed/pre-defined (not learned during training) — every token gets the same weight regardless of the prediction task.

Category : ATTENTION BASICS

Points : 200

Q2: In basic self-attention (no trainable weights), what mathematical operation is used to measure the similarity between two token embeddings?

A: The dot product. You compute the dot product between the current token embedding and every other token embedding in the sequence, then normalize via softmax.

Category : ATTENTION BASICS

Points : 300

Q3: Why do we divide by the square root of d_k in scaled dot-product attention?

A: To prevent the dot products from getting too large, which would push softmax outputs toward 0 or 1 and cause vanishing gradient problems during training.

Category : ATTENTION BASICS

Points : 400

Q4: What is the key difference between self-attention and cross-attention?

A: Self-attention compares tokens within the same sequence to each other. Cross-attention compares tokens from one sequence (or modality) against tokens from a different sequence or modality (e.g., text tokens querying image features).

Category : ATTENTION BASICS

Points : 500

Q5: In Bahdanau (2014) attention for machine translation, name the five key steps of the attention mechanism.

A: 1) Encode the input with a bidirectional RNN. 2) Score: compare the decoder hidden state to each encoder hidden state. 3) Softmax the scores into attention weights. 4) Compute a context vector as the weighted sum of encoder hidden states. 5) Concatenate context with the decoder hidden state to predict the next token.

Category : QKV & TRANSFORMERS

Points : 100

Q6: What do Q, K, and V stand for in the QKV attention framework?

A: Query, Key, and Value. Each is derived by multiplying the input embedding by a separate learned weight matrix (W_q , W_k , W_v).

Category : QKV & TRANSFORMERS

Points : 200

Q7: Why does the QKV framework make attention more flexible than basic dot-product self-attention?

A: The QKV framework introduces trainable weight matrices that project embeddings into new latent spaces before computing dot products. This allows the network to learn an optimal attention rule during training, rather than relying on a fixed/hard-coded similarity measure.

Category : QKV & TRANSFORMERS

Points : 300

Q8: What is the purpose of having multiple "heads" in multi-head attention?

A: Each head has its own set of Q, K, V weight matrices, so each head can learn to attend to different aspects of the input. This lets the model capture more complex patterns by considering different combinations of input features simultaneously.

Category : QKV & TRANSFORMERS

Points : 400

Q9: What are spatial attention and channel attention in the context of CNNs for images?

A: Spatial attention lets the model decide which regions/patches of an image to focus on. Channel attention lets the model decide which feature maps (channels) are most important for a given prediction task.

Category : QKV & TRANSFORMERS

Points : 500

Q10: What is the name of the seminal 2017 paper that introduced the transformer architecture, and what key advantage do transformers have over RNNs?

A: "Attention Is All You Need" by Vaswani et al. Transformers can process all tokens in a sequence in parallel (unlike sequential RNNs), making them faster to train despite being more computationally expensive per step.

Category : ENCODER-DECODER

Points : 100

Q11: In a machine translation encoder-decoder model, why must the decoder be unidirectional?

A: Because the decoder generates output tokens one at a time. At inference, future tokens do not yet exist, so the decoder can only condition on previously generated tokens (left-to-right). The encoder can be bidirectional because the full input sequence is available.

Category : ENCODER-DECODER

Points : 200

Q12: What is the role of causal masking in a transformer decoder?

A: Masking prevents the decoder from "looking ahead" at future tokens in the target sequence during training. This ensures the model only uses previously generated tokens to predict the next one, preserving the autoregressive property.

Category : ENCODER-DECODER

Points : 300

Q13: In a translation transformer using cross-attention, what serves as the Query and what serves as the Key/Value?

A: The Query comes from the attended representation of the target language sequence generated so far (e.g., Spanish). The Key and Value come from the attended representation of the full source language sentence (e.g., English).

Category : ENCODER-DECODER

Points : 400

Q14: Besides MultiHeadAttention, name two other components found inside a transformer block.

A: Residual connections (Add) and Layer Normalization (Norm), plus a feed-forward network (Dense layers). These are often shown as "Add & Norm" blocks in the transformer architecture diagram.

Category : ENCODER-DECODER

Points : 500

Q15: Describe the three places where attention is used during a translation task employing a transformer encoder-decoder.

A: 1) Masked self-attention in the target/decoder sequence (Spanish words output so far). 2) Self-attention in the source/encoder sequence (full English sentence). 3) Cross-attention comparing the decoder representation (Query) against the encoder representation (Key/Value) to find relevant source info

Category : GENERATIVE MODELS

Points : 100

Q16: What is "auto-regression" in the context of a generative language model?

A: Auto-regression means the model generates one token at a time, then feeds its own predicted output back as input to predict the next token. Each new prediction is conditioned on all previously generated tokens.

Category : GENERATIVE MODELS

Points : 200

Q17: How does a CharRNN model differ between training and inference in terms of return_sequences?

A: During training, return_sequences=True so the model predicts one output per token at every position in parallel, computing loss across all positions simultaneously. During inference, return_sequences=False so the model produces only one next-token prediction, which is fed back as input in an auto-re

Category : GENERATIVE MODELS

Points : 300

Q18: Why is a CharRNN considered "decoder-only" — why does it not need an encoder?

A: Because input and output are trivially aligned — the task is simply to predict position $t+1$ from position t . The hidden state already encodes everything from positions 1 through t , so there is no need for a separate encoder to process the input. An encoder is needed when input and output differ in length.

Category : GENERATIVE MODELS

Points : 400

Q19: What does GPT stand for, and what three key ingredients did OpenAI combine to build the GPT models?

A: Generative Pre-trained Transformer. The three ingredients are: i) vast amounts of highly varied text data, ii) a generative (autoregressive) language architecture, and iii) transformers. Plus RLHF (reinforcement learning from human feedback).

Category : GENERATIVE MODELS

Points : 500

Q20: With transformers and enough pre-training, we no longer need the encoder-decoder architecture even for translation. Explain why the encoder-decoder split was originally necessary and what changed.

A: The encoder-decoder split compensated for RNN limitations — RNN hidden states degrade over long sequences, so the encoder saved all input hidden states and attention let the decoder look back. With transformers, self-attention can access all positions directly without degradation, so a single decoder

Category : BERT & LLMS

Points : 100

Q21: What does BERT stand for, and what self-supervised training technique does it use?

A: Bidirectional Encoder Representations from Transformers. BERT masks 15% of words at random in a sequence and trains the model to predict the masked words from the surrounding context (similar idea to Word2Vec but with transformers).

Category : BERT & LLMS

Points : 200

Q22: What is the key advantage of BERT embeddings over Word2Vec embeddings?

A: BERT produces context-dependent (dynamic) embeddings — the same word gets a different vector representation depending on the surrounding words. Word2Vec produces static (generic) embeddings where each word always has the same vector regardless of context.

Category : BERT & LLMS

Points : 300

Q23: Name two ways to obtain a sentence-level embedding from a BERT model.

A: 1) Mean pooling: average the contextual word embeddings from after the MultiHeadAttention layers across all tokens. 2) Use the CLS token embedding — a special [CLS] context token prepended to every input whose output representation is trained to capture the meaning of the entire sequence.

Category : BERT & LLMS

Points : 400

Q24: Name the three major problems with LLMs discussed in class.

A: 1) Hallucination — they produce plausible but not necessarily truthful text. 2) Prompt brittleness — small rewording changes can dramatically change the output. 3) Memorization — hard to tell if benchmark performance reflects genuine reasoning or just recall of training data.

Category : BERT & LLMS

Points : 500

Q25: How does Wav2Vec create audio embeddings, and what self-supervised technique does it borrow from NLP?

A: Wav2Vec first obtains 1D convolutional feature maps from chunks of raw audio, yielding sequences of feature maps. It then randomly masks a subset of these feature maps and trains a transformer to predict them from the visible ones — the same masked prediction idea used by BERT for text.